Dual-Phase Framework for Few-Shot Hyperspectral Image Classification With Spatiospectral Masked Autoencoder and Episode Training

Wijayanti Nurul Khotimah, Mohammed Bennamoun, *Senior Member, IEEE*, Farid Boussaid, Lian Xu, and Ferdous Sohel, *Senior Member, IEEE*

***Abstract*— This article introduces a two-phase learning approach for hyperspectral image (HSI) classification using few-shot learning (FSL). For the first phase, we present a novel spatiospectral masked autoencoder (ssMAE)—an advanced self-supervised learner. For the ssMAE backbone network, we designed a transformer encoder–decoder network, where we replaced the linear layer that is used as the initial feature embedding with a 3-D convolutional layer to better extract local spectral–spatial features from 3-D visible sub-patches. By tapping into vast unlabeled data, the ssMAE learns general HSI features. In the second phase, the ssMAE encoder is fine-tuned to extract discriminative features for classification using the few-shot labeled training samples. This is achieved through a unique hybrid episode learning method that integrates the ssMAE encoder in a prototypical network (PN). We innovate with a mix of global and local prototypes (combined global–local (CGL) prototype) to refine label predictions. This technique maximizes data usage, focuses on specific samples, and mitigates issues from subpar episodes. Tested on three HSI datasets, our approach outperforms alternative few-shot methods. The code will be made publicly available at https://github.com/Weejaa04/SSMAE.**

***Index Terms*— Combination of global and local prototypes, few-shot learning (FSL), hyperspectral image (HSI) classification, hybrid episode learning, prototypical network (PN), spatiospectral masked autoencoder (ssMAE).**

Received 8 November 2023; revised 6 April 2024, 15 July 2024, 21 October 2024, and 2 February 2025; accepted 4 February 2025. Date of publication 13 February 2025; date of current version 4 March 2025. This work was supported in part by the Grains Research and Development Corporation under Grant UWA2002-003RTX and in part by Australian Research Council (ARC) under Grant DP210101682. *(Corresponding author: Wijayanti Nurul Khotimah.)*

Wijayanti Nurul Khotimah is with the Department of Informatics, Institut Teknologi Sepuluh Nopember (ITS), Kampus ITS, Surabaya 60111, Indonesia, and also with the Department of Computer Science and Software Engineering, The University of Western Australia, Perth, WA 6009, Australia (e-mail: wijayanti@if.its.ac.id).

Mohammed Bennamoun and Lian Xu are with the Department of Computer Science and Software Engineering, The University of Western Australia, Perth, WA 6009, Australia (e-mail: mohammed.bennamoun@uwa.edu.au; lian.xu@uwa.edu.au).

Farid Boussaid is with the Department of Electrical, Electronic and Computer Engineering, The University of Western Australia, Perth, WA 6009, Australia (e-mail: farid.boussaid@uwa.edu.au).

Ferdous Sohel is with the School of Information Technology, Murdoch University, Perth, WA 6150, Australia (e-mail: f.sohel@murdoch.edu.au).

Digital Object Identifier 10.1109/TGRS.2025.3541263

NOMENCLATURE

N	Number of categories.
K	Number of training samples per category.
n	Size of input patch.
B	Number of bands.
sa	Spatial dimension of mask.
sp	Spectral dimension of mask.
r	Mask ratio.
D	Embedding size.
L	Depth of encoder.
P	Depth of decoder.
T	Spectrum of angles $(0°, 90°, 180°, 270°)$.
X_{tr}	All training samples.
$f_\Theta(.)$	Pretrained ssMAE encoder (feature extractor).
X_{aug}	All augmented instances.
Y_{aug}	Label of corresponding augmented instances.
M	Number of classes in each episode.
S	Support set.
S^k	Support samples of the kth class in the current episode.
Q	Query set.
Q^k	Query samples of the kth class in the current episode.
$\boldsymbol{Q}$	Covariance matrix.
$\boldsymbol{Q}_k$	Covariance matrix corresponding to the kth class of current episode.
nEpisode	Number of episodes.
C_{ep}	Classes in current episode.
C_{all}	All available classes in the dataset.
(X_s, Y_s)	Corresponding instances and labels of the support set.
(X_q, Y_q)	Corresponding instances and labels of the query set.
$G(\mu)$	Global prototypes mean.
$G(\boldsymbol{Q})$	Global prototype covariance.
$g(\mu_c)$	Global mean of class c (the mean of embedding feature of all X_{aug} within class c).
$G(\boldsymbol{Q}_c)$	Global class covariance (the covariance of all X_{aug} within class c).
X^c_{aug}	Samples from X_{aug} with label c.

1558-0644 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 23:06:40 UTC from IEEE Xplore. Restrictions apply.

Σ_c	Intraclass covariance matrices.
$L(\mu)$	Local prototypes mean.
$L(\boldsymbol{Q})$	Local prototypes covariance.
$l(\mu_k)$	Mean of the local prototype of a class k.
$l(\boldsymbol{Q}_k)$	Covariance of the local prototype of a class k.
Σ_k	Interclass covariance matrices derived from the support sample.
Σ_S	Intraclass covariance matrices derived from the support sample.
$h\mu_k^c$	Similarity score between mean of the local and global prototypes for each class.
$h\boldsymbol{Q}_k^c$	Similarity score between covariance of the local and global prototypes for each class.
$H\mu_k$	Mean similarity score.
$H\boldsymbol{Q}_k$	Covariance similarity score.
Norm$(h\mu_k^j)$	Normalized value of.
Norm$(h\boldsymbol{Q}_k^c)$	Normalized value of.
$p\mu_k^c$	Probability distribution of Norm$(h\mu_k^c)$ that reflect the degree of consistency of the mean.
$p\boldsymbol{Q}_k^c$	Probability distribution of Norm$(h\boldsymbol{Q}_k^c)$ that reflect the degree of consistency of the covariance.

I. Introduction

HYPERSPECTRAL imaging has emerged as a transformative technology across diverse fields, including precision agriculture, environmental monitoring, mineral exploration, and defense [1]. Unlike conventional RGB or multispectral imaging, hyperspectral sensors capture hundreds of narrow, contiguous spectral bands, enabling the discrimination of materials based on their unique spectral signatures. This rich spectral–spatial information has proven invaluable for tasks such as crop stress detection, pollutant mapping, and camouflage identification. However, the high dimensionality of hyperspectral data, coupled with the scarcity of labeled training samples, poses significant challenges for automated classification systems.

Deep learning (DL) models, particularly convolutional neural networks (CNNs), have transformed hyperspectral image (HSI) classification by effectively capturing hierarchical spectral–spatial features [1], [2], [3]. However, their performance heavily depends on the availability of large annotated datasets, which are often scarce in real-world applications due to the labor-intensive and expert-driven nature of hyperspectral labeling [4]. For example, annotating agricultural HSIs requires agronomic expertise to differentiate subtle spectral variations between crop varieties, nutrient deficiencies, or disease stress stages. Such annotations demand both domain knowledge and extensive manual effort. Consequently, models trained on limited labeled data often suffer from overfitting, poor generalization, and high sensitivity to variations in imaging conditions, such as changes in illumination, sensor noise, or occlusions [5], [6]. These challenges become even more pronounced in few-shot learning (FSL) scenarios, where only a handful of labeled examples per class are available. Such constraints frequently arise in critical applications, including rapid damage assessment in disaster response, early-stage disease diagnosis from medical imaging, or the identification of rare and endangered species in ecological monitoring.

To address these challenges, FSL paradigms have gained traction, focusing on enabling models to generalize effectively from minimal labeled data. Early FSL approaches for HSI classification adopted lightweight CNNs [7], [8], [9], [10] to reduce parameter counts and mitigate overfitting. While effective in simple scenarios, shallow networks struggle to capture the intricate spectral–spatial interactions inherent in HSIs. Cross-domain few-shot learning (CFSL) methods improved generalization by transferring knowledge from labeled source domains (e.g., other HSI datasets) to target domains. However, domain shifts caused by differences in sensors, resolutions, or environmental conditions often degrade performance. For example, a model pretrained on airborne HSIs may fail when applied to satellite-based data due to variations in spatial resolution and radiometric calibration. Recent efforts mitigate domain shifts via graph-based alignment or contrastive learning [11], [12] but still rely on labeled source data, which is impractical in many applications.

Self-supervised learning (SSL) offers an alternative by pretraining models on pretext tasks without human annotations [13]. Masked autoencoders (MAEs) which reconstruct randomly masked portions of input data, have shown promise in HSI processing. However, existing HSI-focused MAEs employ isolated spectral or spatial masking strategies [14], [15], neglecting the intrinsic spectral–spatial correlations critical for accurate classification. For instance, masking spectral bands alone fails to leverage spatial context (e.g., texture patterns), while spatial masking overlooks spectral continuity. Furthermore, transformer-based MAEs often use linear embeddings ill-suited for extracting local spectral–spatial features, limiting their ability to model fine-grained HSI structures.

In parallel, prototypical networks (PNs) have become a cornerstone of FSL, representing classes as prototypes (e.g., feature means). Extension of PN incorporates covariance metrics to model intraclass distributions [4]. Yet, conventional PNs rely on episodic training, where randomly sampled support/query sets may contain outliers or noisy samples, destabilizing prototype estimation.

To address these limitations, we propose a novel dual-phase framework combining SSL pretraining and hybrid episode-based fine-tuning. Our key contributions include the following.

1) *Spatiospectral MAE (ssMAE):* A self-supervised pretraining architecture that integrates 3-D convolutions with transformers. Unlike conventional spectral-only or spatial-only masking, ssMAE applies joint spectral–spatial masking, compelling the model to reconstruct missing data by leveraging cross-dimensional correlations. The use of a 3-D convolutional embedding layer instead of linear projections in standard transformers enhances local feature extraction from HSI sub-patches, leading to more robust spectral–spatial representations.
2) *Combined Global–Local (CGL) Prototype Learning:* A hybrid episode-based fine-tuning strategy that combines global prototypes (aggregated from all labeled data) with

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 23:06:40 UTC from IEEE Xplore. Restrictions apply.

local episode-specific prototypes. This fusion improves stability by mitigating the impact of noisy episodes while maximizing data efficiency, which is critical when labeled samples are scarce.

3) *Target Domain Independence:* Unlike cross-domain FSL approaches, our framework operates exclusively within the target domain, eliminating reliance on labeled source data. By leveraging unlabeled HSI samples for pretraining, the model learns domain-specific feature distributions, enhancing adaptability to downstream classification tasks.
4) *Comprehensive Benchmarking:* The framework is evaluated on four benchmark HSI datasets: Pavia University (PU), Indian Pines (IP), Salinas (SA), and Houston (HO) under N-way-five-shot settings. Experimental results demonstrate state-of-the-art performance, while ablation studies confirm the effectiveness of joint spectral–spatial masking and CGL prototypes. Computational analyses highlight the efficiency of the dual-phase learning approach.

II. Related Work

This section reviews key advancements in FSL for hyperspectral imaging, SSL for spectral–spatial feature extraction, and prototype-based metric learning.

A. Few-Shot Learning in Hyperspectral Imaging

FSL aims to generalize from limited labeled examples, making it particularly relevant for HSI classification where annotations are scarce. Early FSL approaches for HSIs sought to mitigate overfitting using shallow CNN-based architectures to reduce model complexity and minimize trainable parameters [16]. However, these models struggled to capture the intricate spectral–spatial relationships in hyperspectral data, limiting classification accuracy.

To improve generalization, cross-domain FSL approaches attempted to transfer knowledge from labeled source datasets (SDs) to target datasets (TDs) with limited labeled samples. However, domain shifts due to variations in sensor types, spatial resolution, and atmospheric conditions often degrade performance [17], [18], [19]. Recent solutions, such as graph-based feature alignment and contrastive learning, have attempted to address these shifts [11], [12]. Despite their effectiveness, these methods still require labeled source data, which may not always be available.

B. Self-Supervised Learning for HSI Feature Extraction

SSL pretrains models using pretext tasks to capture the underlying structure of data without relying on human-labeled annotations. This approach addresses the high cost of manual labeling by enabling feature learning directly from unlabeled data. Additionally, SSL mitigates domain shift issues, as it does not require a labeled SD for training [20], [21]. Early SSL methods for HSI classification focused on tasks such as image reconstruction using downscaling–upscaling techniques [22] and MAEs [23].

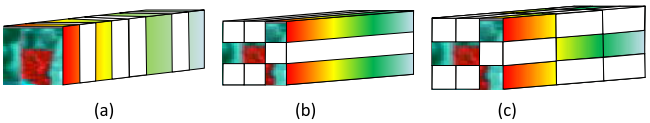


Fig. 1. Illustration of an input segment (sample) with the colorless areas indicating masks. (a) Spectral-only mask sampling as per reference [14]. (b) Spatial-only mask sampling. (c) Our suggested spatiospectral mask sampling.

In MAEs, each input is divided into multiple fixed-size tokens, with a subset of them randomly masked. The model then learns the underlying structure or latent representation of the data by reconstructing the missing tokens. This process relies on an encoder–decoder architecture, where the encoder extracts latent features from the unmasked tokens, and the decoder reconstructs the original input using these representations. Transformer-based encoder–decoder networks, such as vision transformers (ViTs), have been widely used in MAEs due to their strong feature-learning capabilities [14].

In the context of HSI, Ibanez et al. [14] employed an MAE with a vanilla transformer backbone to reduce noise and redundancy. This approach utilizes spectral-only masking, where specific spectral bands within an input patch are concealed [see Fig. 1(a)], forcing the network to reconstruct missing spectral information without spatial context. As a result, this method can lead to a loss of spatial details essential for accurate HSI classification.

In contrast, Huang et al. [15] applied spatial-only masking with a vanilla transformer, forcing the network to reconstruct missing spatial information without spectral context. Furthermore, while vanilla transformers excel at capturing global features, they struggle with extracting fine-grained local structures necessary for effective HSI classification [24].

To tackle these limitations, our approach combines a vanilla transformer with convolutional layers to harness their complementary strengths. Our proposed MAE replaces the initial linear embedding layer in the vanilla transformer with a 3-D convolution layer, enabling more effective extraction of local spectral–spatial features from each visible token. To ensure comprehensive feature learning, our approach applies masking across both spectral and spatial domains, preventing the complete occlusion of any single band or spatial region, as shown in Fig. 1(c). By reconstructing the masked input using spectral and spatial correlations from neighboring visible sub-patches, our method preserves critical spectral–spatial information and overcomes the limitations of previous spectral-only and spatial-only masking strategies.

C. Prototypical Networks and Episode Training

PNs represent classes as prototypes and classify query samples based on their proximity to these prototypes. A specific variant, called PN with a mean (PNM), constructs class prototypes by averaging features from multiple labeled samples of the same class. This approach offers a simple yet effective method for creating representative prototypes. The PNM approach was used in [25], wherein labeled training samples from SD were used to train the feature extractor.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 23:06:40 UTC from IEEE Xplore. Restrictions apply.

Subsequently, a limited number of labeled training samples from TD were used to generate the class prototype during the testing process. However, not leveraging the vital information contained in the sparse labeled samples from TD for training the feature extractor can degrade the model's performance. To tackle this challenge, Zhang et al. [26] improved PN in [25] using the sparse labeled samples from TD for both training the feature extractor and generating the class prototype during the testing process. The PNM in [25] and [26] used the Euclidean distance metric to compute the distance between a sample and its class prototype.

However, the works in [27], [28], and [29] highlighted that the Euclidean metric is not sensitive to the within-class distribution. Consequently, Xi et al. [4] adopted the Mahalanobis distance metric to more accurately represent the within-class sample distributions and integrated the class-covariance metric into the class prototype. In the context of this article, this variant of PN is referred to as the PNM and covariance (PNMC).

To train the feature extractor, most studies on PN [4], [25], [26], [30], [31] employ an episode learning strategy, commonly used in FSL. In this method, training samples are organized into a series of learning problems (referred to as mini-batches or episodes). Every episode consists of a limited number of training samples (support set) and validation samples (query set), mirroring the few-shot scenario task [32]. In this learning strategy, the feature extractor is updated based on the error produced by the prototype of the current episode (local prototype) on the query set's predictions. However, a problem emerges when the current episode is of subpar quality, for instance, if the support and query sets contain noise or outliers. Updating the feature extractor based on errors produced by such samples can potentially deteriorate its overall performance. Additionally, with the episode learning strategy, certain samples might be sampled more frequently than others, potentially affecting the model's balance.

To address these issues, we propose a CGL episodic learning strategy for training the PN. In this approach, we compute both a local prototype, derived from the support set, and a global prototype, aggregated from all training samples, maximizing the use of available data. These prototypes are then combined based on their similarity to form the CGL prototype. The feature extractor is updated based on the error from the CGL prototype's predictions on the query set, ensuring a more stable learning process. This strategy prioritizes informative samples while reducing the influence of poor-quality episodes, as updates are not solely dependent on support and query sets.

III. Proposed Approach

We propose a novel few-shot HSI classification pipeline consisting of a dual-stage learning framework: initial model pretraining through ssMAE, followed by model refinement using CGL-PNM or CGL-PNMC. Fig. 2 outlines the general architecture of this pipeline. Initially, ssMAE helps in extracting general spectral and spatial features from HSI inputs without human-annotated labels. Thereafter, either CGL-PNM or CGL-PNMC is deployed to fine-tune the feature extractor (ssMAE encoder) with the few-shot labeled samples, aiding in mining discriminative spectral and spatial features crucial for few-shot classification. During testing, the labels of the testing samples are predicted by: 1) using the refined ssMAE encoder to extract features of test samples; 2) employing few-shot labeled samples to formulate the network prototype; and 3) adopting the nearest centroid approach to assign labels based on the minimal distance between the test sample features and the network prototype. The nomenclature summarizes all symbols and notations used in this manuscript.

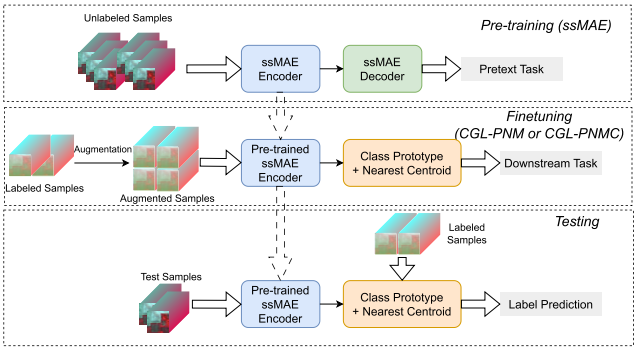


Fig. 2. Overview of the proposed ssMAE-CGL framework, featuring three stages: 1) pretraining the model with ssMAE; 2) fine-tuning of the feature extractor using either CGL-PNMC or CGL-PNM; and 3) testing phase.

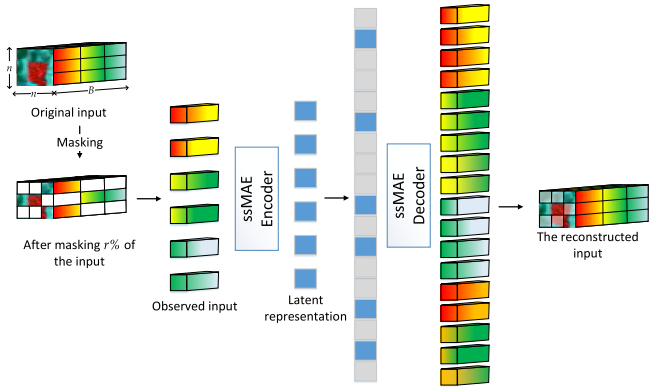


Fig. 3. ssMAE architecture consists of: 1) a masking technique where a large subset of sub-patches in the spectral–spatial dimension is masked randomly; 2) an ssMAE Encoder that operates on the set of unmasked sub-patches, generating encoded sub-patches (latent representation); and 3) an ssMAE Decoder that reconstructs the input using the full set of encoded and masked sub-patches.

A. Pretraining Using the Spatiospectral Masked Autoencoder

The proposed ssMAE, illustrated in Fig. 3, is designed to grasp the inherent structure of HSI input patches without the need for human-annotated labels. This model extends the MAE from [23], aiming for broader spatial and spectral feature learning. It employs a transformer encoder and decoder to reconstruct the initial input from a partial view (unmasked input). More specifically, the ssMAE consists of three main parts.

1) Masking: First, an $n \times n \times B$ sized input is partitioned into S distinct, nonoverlapping sub-patches of dimensions sa $\times$ sa $\times$ sp. Following this, a portion $(r\%)$ of sub-patches are masked, leaving the remaining $(100 - r)\%$ unmasked.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 23:06:40 UTC from IEEE Xplore. Restrictions apply.

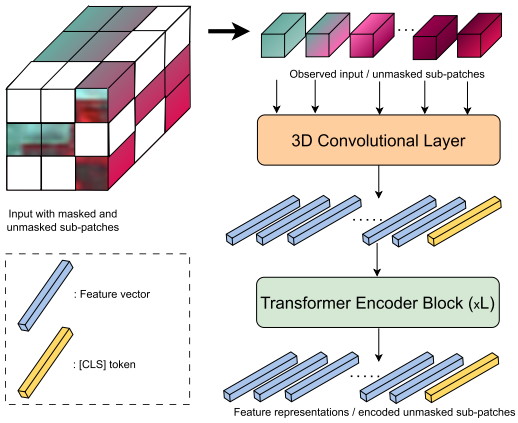


Fig. 4. Architecture of the ssMAE encoder.

2) ssMAE Encoder: Fig. 4 displays our ssMAE encoder's architecture, adopting the standard transformer encoder as per references [33], [34]. Unlike these studies which use a linear projection layer, our ssMAE encoder employs a 3-D convolutional layer for initial input embedding. This layer operates with a kernel and stride size of sa $\times$ sa $\times$ sp, aligning the output channel size with the feature embedding size (D). This choice arises from the 3-D convolutional layer's proven efficiency in extracting local spectral and spatial features [24].

The ssMAE encoder interacts solely with the observed input, consisting of the unmasked sub-patches, and thus operates on a subset of the entire set (for instance, 30% when $r = 70\%$). This results in a sequence length shortened by $r\%$. A larger r value means more sub-patches are omitted, which in turn amplifies redundancy elimination and sequence length reduction. This shortened sequence length mitigates computational and memory demands, facilitating the use of a bigger encoder. Importantly, the index positions of all sub-patches are retained for the following ssMAE decoder reconstruction step. The output of this step includes the general feature representation of the unmasked sub-patches (encoded unmasked sub-patches) and the index positions of all sub-patches.

3) ssMAE Decoder: The ssMAE decoder processes the entire set of sub-patches, including both encoded unmasked and masked sub-patches. Initially, the masked sub-patches are represented by a learned vector, initialized with zeros, having a dimension of $1 \times D$. Index positions are incorporated with the full set of sub-patches to denote their respective placements in the input. Following this, staked blocks of the transformer are applied to predict the masked sub-patches. Finally, both unmasked and estimated masked sub-patches are used for input reconstruction. The loss is measured using mean squared error (mse) between the original and reconstructed inputs, focusing on the pixel levels of the masked sub-patches.

B. Fine-Tuning via CGL-PNM or CGL-PNMC

The ssMAE decoder is exclusively engaged during the pretraining phase for input reconstruction tasks. Post this phase, only the pretrained ssMAE encoder is utilized for downstream tasks to generate general spectral and spatial representations. To enhance the pretrained ssMAE encoder's performance on downstream tasks, we fine-tuned it using our proposed methods that use a hybrid episode strategy in the learning process: CGL-PNMC or CGL-PNM.

Our downstream task is a few-shot classification task termed the N-way-K-Shot problem. In our scenario, we allocate five instances for each class ($K = 5$). For illustration, within the IP dataset, the challenge is dubbed a 16-way-five-shot problem due to IP's 16-class structure.

The episode strategy necessitates a sizable labeled training set. This requirement poses a challenge due to the limited number of available labeled instances. To overcome this obstacle, we augmented our labeled set by employing spatial geometric transformations. This technique is instrumental in generating hyperspectral input patches that closely resemble real-world scenarios [35]. Research has shown that this approach significantly enhances the performance of deep networks [36]. In contrast, alternative augmentation methods, such as Gaussian noise and random cropping, have been found to yield statistically insignificant improvements in deep network performance [17]. HSIs often have various noises [37], and adding Gaussian noise may not provide much additional benefit to the network. The random cropping method, originally designed for RGB and grayscale images, fails to account for the high spectral resolution characteristic of HSI [38].

1) Spatial Geometric Transformation for Data Augmentation: Our input maintains invariance to spatial rotations and reflections since these actions do not alter the label of the center pixel, with its position remaining constant. Therefore, we augment each training sample by applying a series of horizontal rotation operations with a spectrum of angles represented as T, where T includes $\{0°, 90°, 180°, 270°\}$. Given a training sample X_{tr}, we generate its replicas, X_{tr}^{t} where t belongs to T, by rotating it by t degrees. In addition, we perform reflections of $X_{\text{tr}}^{0°}$ and $X_{\text{tr}}^{90°}$ along the x-axis and y-axis, respectively. In total, each sample is expanded into eight samples.

2) Implementation of CGL-PNMC: The procedure of fine-tuning process using the CGL prototype for PNMC, termed CGL-PNMC, is detailed in Algorithm 1. CGL-PNMC refines the pretrained ssMAE encoder $[f_\Theta(.)]$ by making use of the augmented samples $(X_{\text{aug}}, Y_{\text{aug}})$ generated from the augmentation process elaborated in Section III-B1. The inputs for CGL-PNMC comprise X_{aug}, Y_{aug}, C_{all}, M, $f_\Theta(.)$, $|S^k|$, $|Q^k|$, and nEpisode, where $M \leq N$ and $C_{\text{all}} = \{c_1, c_2, c_3, \ldots, c_N\}$.

In every episode, M classes are chosen at random to create a class episode (C_{ep}), where $C_{\text{ep}} = \{k_1, k_2, \ldots, k_M\}$ and C_{ep} being a subset of C_{all}. Following this, the support set (X_s, Y_s) and the query set (X_q, Y_q) are randomly assembled from the augmented samples, ensuring that Y_s belongs to C_{ep}, Y_q belongs to C_{ep}, and the intersection of (X_q, Y_q) and (X_s, Y_s) is empty. The sizes of the support and query sets are given by $M \times |S^k|$ and $M \times |Q^k|$, respectively.

The global prototype $[G(\mu), G(\boldsymbol{Q})]$ is defined using means and covariance. The global class mean $[g(\mu_c)]$ represents the mean of embedding features for all X_{aug} within class cth.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 23:06:40 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 CGL-PNMC

CGL-PNMC $(X_{aug}, Y_{aug}, C_{all}, M, f_\Theta(.), |S^k|, |Q^k|, nEpisode)$

$i \leftarrow 0$
While$(i < nEpisode)$:
Generate $C_{ep} \subseteq C_{all}$
Generate $X_s \subset X_{aug}$ and $Y_s \subset Y_{aug}$ where $Y_s \in C_{ep}$
Generate $X_q \subset X_{aug}$ and $Y_q \subset Y_{aug}$ where $Y_q \in C_{ep}$

Compute the global prototype
While $(c \in C_{all})$:
$g(\mu_c) \leftarrow$ use Eq. 1
$g(\boldsymbol{Q}_c) \leftarrow$ use Eq. 2

Compute the local prototype (episode prototype)
While $(k \in C_{ep})$:
$l(\mu_k) \leftarrow$ use Eq. 4
$l(\boldsymbol{Q}_k) \leftarrow$ use Eq. 5

Compute the similarity between the global and local prototypes
While $(k \in C_{ep})$:
While$(c \in C_{all})$:
$h\mu_k \leftarrow$ use Eq. 7
$h\boldsymbol{Q}_k \leftarrow$ use Eq. 8

Normalize the similarity using Eqs. 9 and 10

Change the normalized similarity into probability distribution using Eqs. 11 and 12

Compute CGL prototype
$g_{CGL}(\mu_k) \leftarrow$ using Eq. 13
$g_{CGL}(\boldsymbol{Q}_k) \leftarrow$ using Eq. 14

Compute the performance of the CGL prototype on query set (X_q) using Eq. 15

Compute the loss using cross-entropy loss (Eq. 16)
Update Θ
$i \leftarrow i + 1$

Similarly, the global class covariance $[g(\boldsymbol{Q}_c)]$ represents the covariance of embedding features for all X_{aug} within class cth. $G(\mu) = \{g(\mu_c)|c \in C_{\text{all}}\}$ is an $N \times D$ matrix that contains the global mean prototype representations for all the classes. On the other hand, $G(\boldsymbol{Q}) = \{g(\boldsymbol{Q}_c)|c \in C_{\text{all}}\}$ is an $N \times D \times D$ matrix containing the global covariance prototype representations for all the classes. When X^c_{aug} represents samples from X_{aug} with label c, the global class mean is calculated using (1). The global class covariance representations for class cth are calculated using (2) as per reference [4], where Σ_c represents intraclass covariance matrices computed from (3), Σ_{aug} represents interclass covariance matrices estimated from all the augmented samples, and $\lambda_c = |X_{\text{aug}}|/(|X_{\text{aug}}| + 1)$

$$g(\mu_c) = \frac{\sum_{x_l \in X^c_{\text{aug}}} f_\Theta(x_l)}{|X^c_{\text{aug}}|} \tag{1}$$

$$g(\boldsymbol{Q}_c) = \lambda_c \Sigma_c + (1 - \lambda_c)\Sigma_{\text{aug}} + I \tag{2}$$

$$\Sigma_c = \frac{\sum_{x_l \in X^c_{\text{aug}}} (f_\Theta(x_l) - g(\mu_c))(f_\Theta(x_l) - g(\mu_c))^T}{|X^c_{\text{aug}}| - 1}. \tag{3}$$

The local prototype $[L(\mu), L(\boldsymbol{Q})]$ is similarly characterized using means and covariance. The mean of the local prototype class $[l(\mu_k)]$ is determined through (4), where X^k_s represents samples from X_s that bear the label k. Following (5) as per reference [4], the covariance of the local prototype class $[l(\boldsymbol{Q}_k)]$ is calculated, where Σ_k and Σ_S represent intraclass and interclass covariance matrices derived from all the support samples, with $\lambda_k = |S|/(|S| + 1)$. The mean of the local prototype for all classes in this episode is organized into a matrix $L(\mu) = \{l(\mu_k)|k \in C_{\text{ep}}\}$, of dimensions $M \times D$, while the covariance of the local prototype for all classes in this episode is organized into a matrix $L(\boldsymbol{Q}) = \{l(\boldsymbol{Q}_k)|k \in C_{\text{ep}}\}$, of dimensions $M \times D \times D$

$$l(\mu_k) = \frac{\sum_{x_l \in X^k_s} f_\Theta(x_l)}{|S^k|} \tag{4}$$

$$l(\boldsymbol{Q}_k) = \lambda_k \Sigma_k + (1 - \lambda_k)\Sigma_S + I \tag{5}$$

$$\Sigma_k = \frac{\sum_{x_l \in X^k_s} (f_\Theta(x_l) - l(\mu_k))(f_\Theta(x_l) - l(\mu_k))^T}{|S^k| - 1}. \tag{6}$$

The similarity score between the mean of the local and global prototypes for each class $(h\mu^c_k)$ is computed using (7), and the similarity score between the covariance of the local and global prototypes for each class $(h\boldsymbol{Q}^c_k)$ is computed using the following equation:

$$h\mu^c_k = -\|l(\mu_k) - g(\mu_c)\|_2, \quad c \in C_{\text{all}},\ k \in C_{\text{ep}} \tag{7}$$

$$h\boldsymbol{Q}^c_k = -\|l(\boldsymbol{Q}_k) - g(\boldsymbol{Q}_c)\|_2, \quad c \in C_{\text{all}},\ k \in C_{\text{ep}}. \tag{8}$$

The mean score similarity $(H\mu_k = [h\mu^1_k, h\mu^2_k, \ldots, h\mu^N_k]^T)$ and covariance score similarity $(H\boldsymbol{Q}_k = [h\boldsymbol{Q}^1_k, h\boldsymbol{Q}^2_k, \ldots, h\boldsymbol{Q}^N_k]^T)$ are then normalized to a range of 0–1 using the following equations:

$$\text{Norm}\left(h\mu^c_k\right) = \frac{h\mu^c_k - \min(H\mu_k)}{\max(H\mu_k) - \min(H\mu_k)} \tag{9}$$

$$\text{Norm}\left(h\boldsymbol{Q}^c_k\right) = \frac{h\boldsymbol{Q}^c_k - \min(H\boldsymbol{Q}_k)}{\max(H\boldsymbol{Q}_k) - \min(H\boldsymbol{Q}_k)}. \tag{10}$$

Next, the normalized similarity is transformed into a probability distribution that reflects the degree of consistency using (11) and (12). If the probability is high (close to 1), the change is minor. However, if the probability is low (close to 0), the change is significant

$$p\mu^c_k = \frac{\text{Norm}\left(h\mu^c_k\right)}{\sum_{c=1}^{c=N} \text{Norm}\left(h\mu^c_k\right)} \tag{11}$$

$$p\boldsymbol{Q}^c_k = \frac{\text{Norm}\left(h\boldsymbol{Q}^c_k\right)}{\sum_{c=1}^{c=N} \text{Norm}\left(h\boldsymbol{Q}^c_k\right)}. \tag{12}$$

The CGL prototype is derived by merging the global prototype of each class in C_{ep} [represented by $g(\mu_k)$ and $g(\boldsymbol{Q}_k)$]

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 23:06:40 UTC from IEEE Xplore. Restrictions apply.

with the probability distribution of the mean (designated as $P\mu_k = [p\mu_k^1, p\mu_k^2, p\mu_k^3, \ldots, p\mu_k^N]$) and the covariance (given by $P\boldsymbol{Q}_k = [p\boldsymbol{Q}_k^1, p\boldsymbol{Q}_k^2, p\boldsymbol{Q}_k^3, \ldots, p\boldsymbol{Q}_k^N]$). These calculations are detailed in (13) and (14). Essentially, the CGL prototype refines the global prototype using the local prototype based on their similarity

$$g_{\mathrm{CGL}}(\mu_k) = g(\mu_k)P\mu_k \tag{13}$$

$$g_{\mathrm{CGL}}(\boldsymbol{Q}_k) = g(\boldsymbol{Q}_k)P\boldsymbol{Q}_k. \tag{14}$$

The similarity between the query samples and the CGL prototype is determined using the Mahalanobis distance, as outlined in (15). Following this, the query loss is calculated based on the cross-entropy loss, comparing the predictions with the actual labels of the query set, as specified in (16). The loss is then utilized to refine Θ

$$\mathrm{pred}_i^q = \underset{k}{\mathrm{argmin}}\Big\{\big(f_\Theta(x_i^q) - g_{\mathrm{CGL}}(\mu_k)\big)^T \times g_{\mathrm{CGL}}(\boldsymbol{Q}_k)\big(f_\Theta(x_i^q) - g_{\mathrm{CGL}}(\mu_k)\big)\Big\} \tag{15}$$

$$L_i^q = \mathrm{CE}\big(y_i^q, \mathrm{pred}_i^q\big) \tag{16}$$

$$\mathrm{pred}_i^q = \underset{k}{\mathrm{argmin}}\Big\{\big(f_\Theta(x_i^q) - g_{\mathrm{CGL}}(\mu_k)\big)^T \times \big(f_\Theta(x_i^q) - g_{\mathrm{CGL}}(\mu_k)\big)\Big\}. \tag{17}$$

3) Implementation of CGL-PNM: The fine-tuning steps using CGL-PNM and CGL-PNMC are the same. However, they differ in the formula used to compute the prototypes (Global, Local, and CGL prototypes). The detailed formulas for CGL-PNM are as follows.

1) CGL-PNM only has global and local mean prototypes, which formulas are the same as (1) and (4).
2) In CGL-PNM, we compute the similarity only between the global and local mean prototypes, as presented in (7).
3) The similarity between the query set and the CGL prototype is computed using the Euclidean distance, as shown in (17).

C. Testing Approach for CGL-PNMC and CGL-PNM

For the testing phase, we opted for the nearest centroid method due to its demonstrated superiority over both k nearest neighbours (k-NN) and soft assignments, as cited in [32]. Here is how we proceeded: 1) we made use of the fine-tuned feature encoder, represented as $[f_\Theta(.)]$, to extract important features from both the labeled training and unlabeled test samples; 2) during this process, the labeled training samples, consisting of five labeled samples for each class, were used to construct the network prototype; and 3) the prediction of each test sample was made by identifying the closest distance between the sample's features and the network prototype.

For the CGL-PNMC model: 1) the network prototype is derived from (1) and (2) and 2) the measurement of the distance between the features from the test sample and the said network prototype is calculated via (15). **Conversely, in the CGL-PNM model:** 1) the network prototype is determined solely using (1) and 2) the distance between the extracted features of the test sample and the network prototype is determined through (17).

IV. Experiments and Analysis

A. Datasets and Evaluation Measures

To assess the performance of our approach, we conducted experiments on three well-known benchmark datasets: PU, IP, SA, and HO. We carried out five independent runs for each test and reported the mean and standard deviation of the results in terms of overall accuracy (OA), average accuracy (AA), and Kappa (K). The specific categories for these three datasets are detailed in Table I.

1) *PU:* This dataset was acquired using a Reflective Optics Spectrographic Imaging System (ROSIS) sensor. With a spatial resolution of 1.3 m/pixel and 103 spectral bands, its dimension is 610 × 340 pixels. However, only 42 776 pixels have ground-truth labels, which are spread into nine categories.
2) *IP:* Acquired using an Airborne Visible/Infrared Imaging Spectrometer (AVIRIS) sensor, this dataset covers a 0.4–2.5-μm wavelength range. After removing bands affected by water absorption, 200 bands were retained. Its dimension is 145 × 145 pixels, with a spatial resolution of 20 m/pixel, and consists of 16 categories spread over 10 249 pixels.
3) *SA:* Like IP, SA's data was acquired with the AVIRIS sensor and covers the 0.4–2.5-μm wavelength range. Removing the bands affected by water absorption leaves 204 bands. Compared to IP, SA has a higher spatial resolution of 3.7 m/pixel. Its dimension is 512 × 217 pixels and includes 16 categories.
4) *HO:* The HO dataset was collected by the NSF-funded Centre for Airborne Laser Mapping (NCALM) and was first used for the 2013 IEEE GRSS data fusion contest [39]. The data was captured over the University of Houston and its surrounding area on June 23, 2012 between 17:37:50 and 17:39:50 UTC, with an average sensor height of 5500 ft above ground. The data comprises 144 bands, 349 × 1905 pixels, with 15 029 pixels labeled across 15 different classes, detailed in Table I.

B. Experimental Settings

Our proposed approach involves a two-phase learning process: initial learning through pretraining, followed by fine-tuning. Following prior works [14], [20], [21] that utilize an SSL framework for pretraining, we used the entire dataset (without labels) to pretrain the ssMAE in this phase, where the model learns intrinsic features through sample masking and reconstruction. Details of the parameters used for pretraining are provided in Table II.

Based on insights from [23], a smaller decoder still provides satisfactory downstream performance when the encoder is fine-tuned for that downstream task. Thus, to optimize resource usage and training time, we have opted for a smaller decoder. In our experiments, L is 12 and P is 3. We have chosen a patch size (n) of 9 following the approach in [4], [11], [31], and [40]. The mask ratio (r), embedding dimensions (D), and mask size are set based on our observation that has been depicted in Table III. Table III demonstrates that the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 23:06:40 UTC from IEEE Xplore. Restrictions apply.

TABLE I

CATEGORY BREAKDOWN FOR PU, IP, AND SA DATASETS (THE COLORS CORRESPOND TO THE LABELS USED IN THE VISUALS OF SECTION IV-E)

PU dataset		IP dataset		SA dataset		HO dataset	
C1	Asphalt	C1	Alfafa	C1	Brocoli_green_weeds_1	C1	Healthy grass
C2	Meadows	C2	Corn-notil	C2	Brocoli_green_weeds_2	C2	Stressed grass
C3	Gravel	C3	Corn-mintill	C3	Fallow	C3	Synthetic grass
C4	Trees	C4	Corn	C4	Fallow_rough_plow	C4	Trees
C5	Painted metal sheets	C5	Grass-pasture	C5	Fallow_smooth	C5	Soil
C6	Bare soil	C6	Grass-trees	C6	Stubble	C6	Water
C7	Bitumen	C7	Grass-pasture-mowed	C7	Celery	C7	Residential
C8	Self-Blocking Bricks	C8	Hay-windrowed	C8	Grapes_untrained	C8	Commercial
C9	Shadows	C9	Oats	C9	Soil_vinyard_develop	C9	Road
		C10	Soybean-notil	C10	Corn_senesced_green_weeds	C10	Highway
		C11	Soybean-mintill	C11	Lettuce_romaine_4wk	C11	Railway
		C12	Soybean-clean	C12	Lettuce_romaine_5wk	C12	Parking lot 1
		C13	Wheat	C13	Lettuce_romaine_6wk	C13	Parking lot 2
		C14	Woods	C14	Lettuce_romaine_7wk	C14	Tennis court
		C15	Building-Grass-Tress	C15	Vinyard_untrained	C15	Running track
		C16	Stone-Steel-Towers	C16	Vinyard_vertical_trellis		

TABLE II

EXPERIMENTAL SETTING FOR PRETRAINING. THE DATA FOR PRETRAINING IS THE ENTIRE RESPECTIVE DATASET WITHOUT LABELS

Parameter	PU	IP	SA	HO
Depth of encoder (L)	12			
Depth of decoder (P)	3			
Size of input patch (n)	9			
Number of bands (B)	200	103	204	144
Transformer head	3			
Mask ratio (r)	70%			
Embedding size (D)	96	192	192	192
Mask size	3x3x10	3x3x20	3x3x20	3x3x20
Optimizer	Adam optimizer			
Learning rate	0.00015			
Batch size	512	256	256	256
Epoch	1000			

TABLE III

IMPACT OF SEVERAL PARAMETERS (MASK RATIO, EMBEDDING SIZE, AND MASK SIZE) TO THE OA

Parameters	Value	PU	IP	SA
Mask ratio	60%	81.65±3.91	66.5±3.44	86.12±2.75
	70%	**90.62±3.75**	**75.82±1.53**	**87.72±4.43**
	80%	88.30±0.66	73.26±2.60	84.59±0.31
Embedding size	96	**90.62±3.75**	75.82±1.53	87.72±4.43
	192	85.06±4.86	**76.40±4.28**	**87.83±4.09**
Mask size	3x10	**90.62±3.75**	76.40±4.20	87.83±4.09
	3x20	89.67±3.08	**80.89±2.10**	**92.81±0.88**

TABLE IV

EXPERIMENTAL SETTING FOR FINETUNING. THE DATA FOR FINE TUNING IS FIVE LABELED SAMPLES PER CATEGORY

Parameter	PU	IP	SA	HO
Number of categories (N)	9	16	16	15
Number of classes in each episode (M)	9	16	16	15
Number of training samples per class (K)	5			
Support samples per class(S^k)	5			
Query samples per class (Q^k)	15			
$nEpisode$	200			
$Optimizer$	Adam optimizer			
Learning rate	0.0003			
Batch size	32			

optimal mask ratio for all datasets is 70%. Reducing the mask ratio to 60% leads to a decrease in OA. Notably, the optimal mask ratio falls below the suggested mask ratio for video (i.e., 90%) [41] yet exceeds the suggested ratio for language (i.e., 15%) [42]. This finding aligns with the hypothesis presented in [23], which said that the information redundancy within the data impacts the optimal mask ratio. Given that HSI data exhibit higher redundancy than language data, a larger mask ratio is required. In terms of embedding size and mask size, the optimal values for PU are 96 and $3 \times 10 \times 10$, respectively, while for IP and SA, the optimum values are 192 and $3 \times 20 \times 20$. Considering that the embedding size and mask size influence the capacity of a network, it is hypothesized that IP and SA datasets, which contain twice the number of bands as PU, require a higher capacity network due to their more complex data.

Subsequently, in the FSL scenario, we fine-tune the model using a small subset of labeled data, specifically five samples per class. These training samples are augmented using spatial geometric transformations as detailed in Section III-B1. We then fine-tune the model using CGL-PNM or CGL-PNMC under the following settings: nEpisode: 200, where N is the number of categories, M is set equal to N, K is set as 5, and Q value: 15. We employed the Adam optimizer with a learning rate adjusted to 0.0003, and the batch size during this phase was set to 32. The parameters used for fine-tuning are also detailed in Table IV, ensuring the model is optimized for specific classification tasks with minimal labeled data.

C. Ablation Study on Mask Shapes

In our work, we propose a novel mask sampling technique. This method involves a combined spatial and spectral mask sampling having dimensions of sa $\times$ sa $\times$ sp. Here, sa is less than n, sp is less than B, and the remainder when n is divided by sa is zero. Since B divided by sp does not yield a zero remainder, we skip the final few bands that do not evenly divide. We assessed the performance of this mask sampling method against the spectral-only and spatial-only mask methods. Consistency was maintained by keeping the same architecture and parameters for all tests. For the spectral-only mask sampling, we chose masks with dimensions $n \times n \times 1$ for PU and $n \times n \times 2$ for IP and SA. On the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 23:06:40 UTC from IEEE Xplore. Restrictions apply.

TABLE V
PERFORMANCE COMPARISON OF DIFFERENT MASK SAMPLING TECHNIQUES ACROSS DIFFERENT DATASETS

Mask sampling	PU	IP	SA
spectral-only	70.64±5.74	50.6±1.41	87.33±2.46
spatial-only	88.25±3.98	79.12±2.1	92.67±1.34
spatiospectral	**90.62±3.75**	**80.89±2.1**	**92.81±0.88**

other hand, spatial-only mask sampling utilized masks sized at $1 \times 1 \times B$ across all datasets.

Table V provides insights on the performance of the three mask sampling methods: spectral-only, spatial-only, and the combined spatial–spectral approach. The data reveals that the combined spatial–spectral mask sampling achieves the best performance. A spectral band is typically bordered by two adjacent bands. Predicting the central band becomes a challenge with spectral-only mask sampling when both adjacent bands are hidden, complicating the pretraining phase. To illustrate, when sampling from a set of ten bands with a 70% masking ratio, only three spectral bands are accessible to predict the other seven, posing a significant challenge.

On the other hand, spatial-only mask sampling exhibits better performance than the spectral-only method. This is attributed to its less demanding pretraining phase. Considering the spatial dimensions, a pixel typically has eight neighboring pixels. Even when 70% of these are masked, the pixel's value can be derived using the remaining two neighbors. Nonetheless, the combined spatial–spectral mask sampling shines, outperforming both the individual methods (i.e., spectral-only and spatial-only mask samplings). Drawing from both spatial and spectral information, it simplifies the pretraining phase significantly.

D. Ablations Study on ss-MAE, CGL-PNMC, and CGL-PNM

In this article, we propose two novel techniques: ssMAE, tailored to capture the broad spectral and spatial characteristics of the data, and CGL, incorporated in PNMC and PNM for fine-tuning to identify specific features of the data. To demonstrate the effectiveness of these modules, we conducted an ablation study on the PU, IP, and SA datasets. To ensure a fair comparison, we maintained consistency using the same architecture for the feature extractor, comprising a 3-D convolutional layer topped with transformer encoder blocks. Table VI displays the outcomes of the ablation study. Additionally, we included information regarding the average execution time for each iteration of every module, expressed in milliseconds (ms), and enclosed the information in parentheses. Please note that we report the time for each module only once per dataset, as the execution time remains consistent for each module within the same dataset.

1) *Impact of ssMAE on Performance:* We examined the influence of ssMAE by comparing the output of a feature extractor using ssMAE to one pretrained using a supervised method. This supervised method involves training the feature extractor with augmented samples. As shown in Table VI, the differences in performance are evident: using ssMAE for pretraining notably boosts the classification output when compared to methods not leveraging ssMAE. Specifically, there are marked performance improvements when comparing R1 to R5 for PU, R8 to R12 for IP, and R15 to R18 for SA. This demonstrates that pretraining with ssMAE enhances the feature encoder robustness, making it more adept for downstream tasks.
2) *Impact of 3-D Convolution on Performance:* We examined the effectiveness of using 3-D convolution to extract the local spectral–spatial features of each token by comparing it with a general visual transformer that employs 2-D convolution, which can only extract spatial features. As shown in Table VII, the MAE that utilizes 3-D convolution significantly outperforms the MAE that employs 2-D convolution, demonstrating the effectiveness of 3-D convolution in extracting the local spectral–spatial features of each token.
3) *Role of Fine-Tuning With Prototype Network:* Here, we analyzed the benefit of fine-tuning in enhancing the robustness of the feature encoder. Without fine-tuning, the ssMAE's feature encoder is used directly for classification. The difference between using and not using fine-tuning can be measured by comparing sets like R2 to R3–R7, R9 to R10–R14, and R16 to R17–R21. Overall, fine-tuning boosts the feature encoder's robustness for the downstream task, yielding better performance than without fine-tuning.

TABLE VI
PERFORMANCE EVALUATION OF OUR PROPOSED APPROACH: SSMAE PRETRAINING AND CGL-PNMC AND CGL-PNM FINE-TUNING ACROSS VARIOUS DATASETS

ID	Dataset	Pre-train (ms)	Fine-tune (ms)	OA (%)
R1	PU	Supervise	CGL-PNMC	71.38±5.88
R2	PU	ssMAE	-	82.57±2.30
R3	PU	ssMAE (590.87)	FCL (0.46)	89.78±3.87
R4	PU	ssMAE	PNMC (0.59)	83.77±4.45
R5	PU	ssMAE	CGL-PNMC (1.59)	90.62±3.75
R6	PU	ssMAE	PNM (0.79)	90.52±3.35
R7	PU	ssMAE	CGL-PNM (0.95)	**91.24±2.89**
R8	IP	Supervise	CGL-PNMC	53.93±2.25
R9	IP	ssMAE	-	67.27±3.07
R10	IP	ssMAE (215.87)	FCL (0.53)	74.89±5.67
R11	IP	ssMAE	PNMC (0.91)	75.56±1.12
R12	IP	ssMAE	CGL-PNMC (2.43)	**80.89±2.10**
R13	IP	ssMAE	PNM (1.07)	73.44±3.91
R14	IP	ssMAE	CGL-PNM (2.00)	77.74±1.80
R15	SA	Supervise	CGL-PNMC	80.38±1.95
R16	SA	ssMAE	-	86.57±0.81
R17	SA	ssMAE (250.61)	FCL (0.77)	89.02±1.43
R18	SA	ssMAE	PNMC (0.93)	90.95±1.65
R19	SA	ssMAE	CGL-PNMC (2.12)	**92.81±0.88**
R20	SA	ssMAE	PNM (1.53)	91.27±1.10
R21	SA	ssMAE	CGL-PNM (1.50)	92.68±0.76

TABLE VII
IMPACT OF 3-D CONVOLUTION LAYER IN EXTRACTING LOCAL SPECTRAL–SPATIAL FEATURES TO THE PERFORMANCE

Dataset	IP		PU	
Conv. Layer	2D	3D	2D	3D
OA	74.32±2.01	**80.89±2.10**	78.37±1.58	**90.62±3.75**
AA	82.54±1.80	**88.84±1.03**	85.04±1.29	**91.67±1.66**
Kappa	71.16±2.20	**78.39±2.31**	72.98±1.72	**87.90±4.64**

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 23:06:40 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
CLASSIFICATION PERFORMANCE (%) OF VARIOUS METHODS ON PU DATASET

Class	DFSL [25]	S-DMM [45]	DCFSL [31]	CFSL [11]	CMFSL [4]	S3Net [46]	CPSRCL [12]	GCC-FSL [40]	Our proposed	
									CGL-PNM	CGL-PNMC
1	73.05±2.78	81.69±13.06	80.78±7.11	85.54±8.39	75.25±4.35	81.69±13.06	*86.91**	79.98±5.97	**88.50±1.85**	87.63±5.01
2	69.82±4.79	73.34±18.75	84.75±9.39	83.76±10.39	86.24±9.81	73.34±18.75	88.94*	*93.96±3.03*	**90.77±7.57**	89.87±9.36
3	84.85±10.19	77.86±19.03	69.63±8.55	63.03±4.45	73.40±10.78	77.86±19.03	80.06*	53.63±5.74	**86.08±7.11**	*83.22±11.72*
4	92.96±2.36	85.52±9.21	93.13±3.34	93.42±4.43	**93.66±3.14**	85.52±9.21	*93.30**	93.23±0.83	91.57±9.29	91.90±8.38
5	99.75±0.27	*99.61±0.64*	**99.88±0.15**	99.09±1.27	99.39±0.96	99.61±0.64	99.40*	98.88±1.37	99.13±0.99	98.98±1.16
6	*83.96±6.04*	82.72±15.25	80.78±12.31	75.89±11.46	83.79±11.05	82.72±15.25	79.22*	73.76±14.45	**96.56±3.78**	96.46±3.00
7	91.88±1.99	**99.43±0.60**	75.23±4.78	78.25±5.38	76.57±4.79	**99.43±0.60**	90.26*	82.38±4.65	90.54±7.45	*91.19±7.05*
8	70.71±4.08	*79.05±16.04*	62.73±5.82	73.36±3.97	75.07±9.94	*79.05±16.04*	86.39*	79.5±10.97	**90.55±4.21**	**90.55±3.18**
9	88.87±6.54	95.92±5.14	**99.64±0.45**	*98.19±2.41*	99.28±0.94	95.92±5.14	99.47*	97.56±1.80	94.44±8.71	95.24±5.95
OA	76.50±1.92	79.45±7.97	82.14±2.41	82.52±4.32	83.59±3.87	79.45±7.97	*87.74**	86.02±1.12	**91.24±2.89**	90.62±3.75
AA	83.98±1.65	86.13±3.18	82.95±1.75	83.39±1.03	84.74±1.47	86.13±3.18	*89.33**	83.66±0.40	**92.02±1.01**	91.67±1.66
K	70.60±2.24	74.26±9.12	76.90±2.73	77.40±4.90	78.81±4.59	74.26±9.12	*83.94**	81.49±1.24	**88.66±3.56**	87.90±4.64

* Variance is not presented in the original CPSRCL paper.

TABLE IX
CLASSIFICATION PERFORMANCE (%) OF VARIOUS METHODS ON IP DATASET

Class	DFSL [25]	S-DMM [45]	DCFSL [31]	CFSL [11]	CMFSL [4]	S3Net [46]	CPSRCL [12]	GCC-FSL [40]	Our proposed	
									CGL-PNM	CGL-PNMC
1	96.96±4.24	80.00±19.39	92.20±6.54	92.68±8.62	97.07±3.18	**100±0**	98.78*	80.49±24.15	*98.05±4.36*	99.51±1.09
2	39.12±6.71	33.59±9.33	38.72±8.93	55.41±11.41	50.50±5.45	*64.62±16.12*	**71.05***	59.07±12.47	60.00±9.55	65.85±11.25
3	66.82±8.25	41.45±14.14	34.67±6.39	48.20±3.10	46.09±14.96	*61.04±18.56*	64.06*	47.03±4.79	57.96±10.98	**64.68±6.63**
4	85.15±3.47	73.62±11.54	61.21±9.53	73.54±13.05	92.33±6.49	*96.12±1.88*	92.03*	70.69±4.27	**97.50±2.75**	96.47±3.40
5	*82.94±7.23*	73.77±15.82	64.77±8.41	75.06±8.54	79.16±7.23	81.72±7.99	**86.19***	80.86±1.63	82.17±7.91	85.06±4.48
6	*94.22±2.62*	89.08±2.97	82.92±4.91	77.38±8.22	80.85±11.37	93.32±9.19	86.90*	92.97±3.90	**94.87±3.84**	94.62±2.89
7	**100±0**	**100±0**	*97.39±3.89*	95.65±7.53	99.13±1.95	**100±0**	**100***	**100±0**	**100±0**	99.13±1.95
8	86.94±2.31	74.12±21.68	73.57±15.17	84.14±10.98	73.40±8.19	80.34±18.81	97.67*	85.73±1.34	*97.59±2.91*	**98.94±1.13**
9	**100±0**	98.67±2.98	**100±0**	**100±0**	**100±0**	**100±0**	**100***	**100±0**	**100±0**	**100±0**
10	*70.66±9.83*	60.37±4.92	52.64±13.71	53.30±6.04	63.39±5.11	67.22±13.79	76.01*	59.73±7.53	**77.83±6.84**	**77.83±3.24**
11	51.29±13.85	44.47±17.00	54.47±10.62	59.65±4.79	49.20±17.80	*71.46±14.37*	73.20*	57.23±1.96	70.85±5.23	**74.96±4.11**
12	62.43±7.27	28.78±16.23	40.89±14.39	42.14±9.44	46.26±7.21	78.71±10.51	56.80*	42.52±21.16	*76.29±5.26*	**84.42±5.77**
13	*97.56±2.23*	96.40±4.98	96.90±1.88	92.30±9.23	96.80±5.22	**99.50±0.61**	96.25*	89±15.56	98.80±2.41	96.50±4.51
14	87.95±1.60	85.92±6.40	81.14±5.20	79.41±11.03	86.46±6.54	*92.63±13.03*	93.21*	79.53±5.06	92.49±5.61	**94.00±4.91**
15	81.45±8.31	48.19±12.33	59.00±13.22	72.18±9.49	68.87±16.31	**90.81±6.15**	64.04*	79.53±4.45	*88.87±10.70*	89.45±6.08
16	99.78±0.48	99.55±1.02	93.41±9.69	**100±0**	**100±0**	*99.77±0.51*	99.43*	99.43±0.81	**100±0**	**100±0**
OA	67.79±2.86	57.03±4.69	58.22±2.73	64.14±1.63	63.19±2.88	77.06 ±3.15	*77.65**	66.18±1.62	77.74±1.80	**80.89±2.10**
AA	81.46±0.85	70.50±2.47	70.24±1.84	75.06±1.88	76.84±1.01	*86.08±2.45*	84.73*	61.89±1.14	87.08±0.89	**88.84±1.03**
K	64.05±2.99	51.97±4.99	52.96±3.03	59.65±1.80	58.80±2.86	74.16 ±3.45	*74.63**	61.89±1.61	74.91±2.01	**78.39±2.31**

* Variance is not presented in the original CPSRCL paper.

TABLE X
CLASSIFICATION PERFORMANCE (%) OF VARIOUS METHODS ON SA DATASET

Class	DFSL [25]	S-DMM [45]	DCFSL [31]	CFSL [11]	CMFSL [4]	S3Net [46]	CPSRCL [12]	GCC-FSL [40]	Our proposed	
									CGL-PNM	CGL-PNMC
1	*99.46±0.56*	98.92±1.60	99.43±0.71	98.84±0.69	98.97±1.28	98.07 ±2.19	99.58*	99.00 ±0.60	**99.67±0.24**	99.44±0.63
2	99.84±0.22	98.04±2.54	95.76±5.42	99.44±0.31	98.44±2.34	94.09±7.13	**100***	99.64±0.20	99.72±0.26	*99.76±0.18*
3	96.79±2.55	88.62±11.72	87.71±10.49	87.83±11.51	91.90±10.85	**99.98±0.04**	96.63*	91.22±7.71	99.93±0.11	*99.91±0.16*
4	97.99±1.33	*98.53±1.45*	99.02±1.02	97.94±1.48	96.28±3.13	95.06±8.02	**99.96***	97.34±2.45	96.4±5.19	96.70±4.24
5	*95.32±3.39*	**96.25±4.14**	94.43±1.87	92.46±4.18	92.13±6.01	95.37±7.28	90.85*	88.08±1.22	95.23±5.06	93.99±5.36
6	98.94±0.61	98.72±1.20	99.24±0.56	98.88±1.05	98.32±1.32	99.60±0.41	**99.99***	98.19±1.78	99.05±1.12	*99.11±0.92*
7	98.45±1.81	98.92±2.18	98.42±2.04	99.62±0.40	97.06±4.24	98.95 ±1.71	**99.85***	*99.78±0.08*	98.76±1.86	99.80±0.21
8	71.56±1.65	74.38±8.49	77.12±4.09	*78.26±7.68*	73.91±5.19	75.61 ±4.51	**83.87***	67.51±5.11	78.47±5.31	78.00±3.95
9	99.22±0.19	98.97±1.60	98.8±1.13	99.11±1.25	97.86±1.61	*99.43±0.82*	**100***	99.52±0.35	99.98±0.03	**100±0.01**
10	90.44±7.38	89.26±6.56	90.79±5.25	86.23±8.29	89.82±9.50	**96.85±3.05**	88.11*	80.97±14.33	95.50±2.15	*95.41±1.83*
11	98.76±0.69	95.09±4.23	94.06±8.76	90.7±12.55	93.28±9.39	**99.51±0.84**	99.39*	98.49±1.22	*98.89±2.38*	98.23±3.85
12	99.35±1.35	98.08±2.90	**99.63±0.34**	98.52±2.26	94.96±10.65	88.76±9.64	98.13*	*98.86±0.83*	96.31±5.28	96.65±4.60
13	**100±0**	*99.54±0.44*	99.34±0.34	98.84±1.00	99.60±0.37	87.53±13.13	98.90*	99.01±0.99	99.19±0.69	99.47±0.33
14	*99.33±0.88*	97.52±1.45	98.08±1.08	98.25±0.82	98.44±0.44	96.00±4.42	98.78*	96.95±1.55	99.57±0.61	**99.62±0.40**
15	78.47±8.44	80.16±4.62	80.27±5.45	78.47±5.96	79.95±3.30	**93.84±5.58**	75.30*	83.99±1.27	*86.26±6.05*	87.79±6.33
16	*98.01±0.82*	89.22±10.85	91.02±4.80	87.96±7.11	86.15±5.51	97.17±2.81	92.92*	87.93±3.69	99.41±0.32	**99.61±0.23**
OA	89.78±1.21	89.67±1.98	90.18±0.80	89.89±1.12	88.95±1.36	*92.11±1.02*	91.64*	88.13±0.50	92.68±0.76	**92.81±0.88**
AA	95.12±0.73	93.77±1.48	93.95±0.34	93.21±0.65	92.94±0.85	94.74±1.47	*95.14**	92.90±0.15	96.39±0.47	**96.47±0.55**
K	88.65±1.35	88.53±2.19	89.10±0.88	88.78±1.22	87.73±1.51	*91.25±1.12*	90.69*	86.85±0.55	91.87±0.84	**92.02±0.97**

* Variance is not presented in the original CPSRCL paper.

4) *Effect of the Hybrid Episode Learning Strategy During Fine-Tuning:* Our focus here was to measure the benefit of our proposed fine-tuning method. This method updates the model considering feedback from both global and local prototypes (CGL prototype) rather than just the local prototype. This approach is applied in both PNMC and PNM. The influence of CGL on PNMC is evident when comparing results like R4 to R5, R11 to R12, and R18 to R19. Using CGL improves results by roughly 5% for PU and IP and about 2% for SA. Similarly, for CGL's effect on PNM, results like R6 to R7, R13 to R14, and R20 to R21 show that CGL boosts performance by about 1% for PU and SA and 4% for IP.

Additionally, when comparing CGL-PNMC with CGL-PNM on the IP and SA datasets, CGL-PNMC tends

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 23:06:40 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
CLASSIFICATION PERFORMANCE (%) OF VARIOUS METHODS ON HO DATASET

Class	S-DMM [45]	DCFSL [31]	CFSL [11]	CMFSL [4]	S3Net [46]	GCC-FSL [40]	Our proposed CGL-PNM	Our proposed CGL-PNMC
1	*89.40±6.15*	89.60±6.48	89.12±7.68	**89.77±7.58**	78.39±14.04	83.71±0.79	84.35±6.70	83.84±5.15
2	**86.65±6.85**	80.90±4.31	69.91±12.69	*81.28±6.23*	75.05±11.87	84.59±8.89	80.00±4.54	80.57±9.24
3	99.28±0.70	97.49±2.82	95.14±2.21	96.94±1.44	**99.51±0.24**	91.98±2.76	*98.04±1.87*	97.89±2.06
4	**93.37±3.86**	90.15±3.65	86.55±3.71	91.61±4.99	86.15±12.50	*91.20±2.86*	82.77±3.87	80.28±4.76
5	88.46±12.80	90.07±4.28	86.69±7.21	92.72±6.54	78.45±15.91	*93.57±1.54*	**94.76±3.76**	94.26±7.79
6	91.56±5.58	83.25±4.35	73.25±11.48	87.63±5.73	*93.69±5.68*	78.13±10.61	**94.13±5.55**	93.88±5.21
7	63.94±11.09	68.93±11.33	53.11±13.83	*82.37±4.37*	60.46±13.18	56.50±13.05	**87.44±5.13**	84.64±4.44
8	**49.70±15.88**	44.78±11.29	37.45±12.40	49.28±10.15	38.18±7.15	43.46±14.78	*49.04±12.75*	47.56±13.11
9	64.09±20.07	57.63±15.28	58.24±16.59	**69.16±11.00**	52.08±11.31	63.96±1.31	69.05±11.28	*66.06±13.02*
10	30.36±10.64	58.76±11.33	54.60±22.42	*63.98±18.15*	54.52±11.14	54.42±32.40	71.31±18.52	**71.88±18.80**
11	57.93±19.86	64.57±2.95	51.50±12.04	61.82±9.76	*72.00±23.89*	50.20±25.70	73.51±11.45	**77.9±15.04**
12	56.40±13.03	**79.46±8.10**	60.47±14.80	71.48±15.89	50.33±20.16	71.22±14.22	*71.32±14.50*	65.62±11.81
13	48.02±22.56	91.25±2.62	70.60±12.64	88.71±2.89	86.77±4.84	86.75±2.89	**91.38±3.06**	*91.12±3.57*
14	95.37±7.11	85.58±9.54	86.10±8.26	98.25±1.71	**99.91±0.21**	95.51±0.33	99.86±0.21	*99.81±0.31*
15	**99.51±0.43**	96.85±2.40	88.64±8.33	97.56±0.87	99.24±0.51	92.75±3.78	*98.47±2.75*	98.41±2.45
OA	71.58±5.59	75.90±3.19	68.19±4.09	*78.68±1.80*	70.02±3.52	72.85±0.18	**79.87±2.06**	78.93±2.02
AA	74.27±5.31	78.62±2.59	70.76±3.61	*81.50±1.35*	74.97±2.68	75.87±0.95	**83.03±1.52**	82.25±1.64
K	69.29±6.08	73.96±3.44	65.65±4.42	*76.96±1.94*	67.66±3.76	70.65±0.18	**78.25±2.23**	77.23±2.18

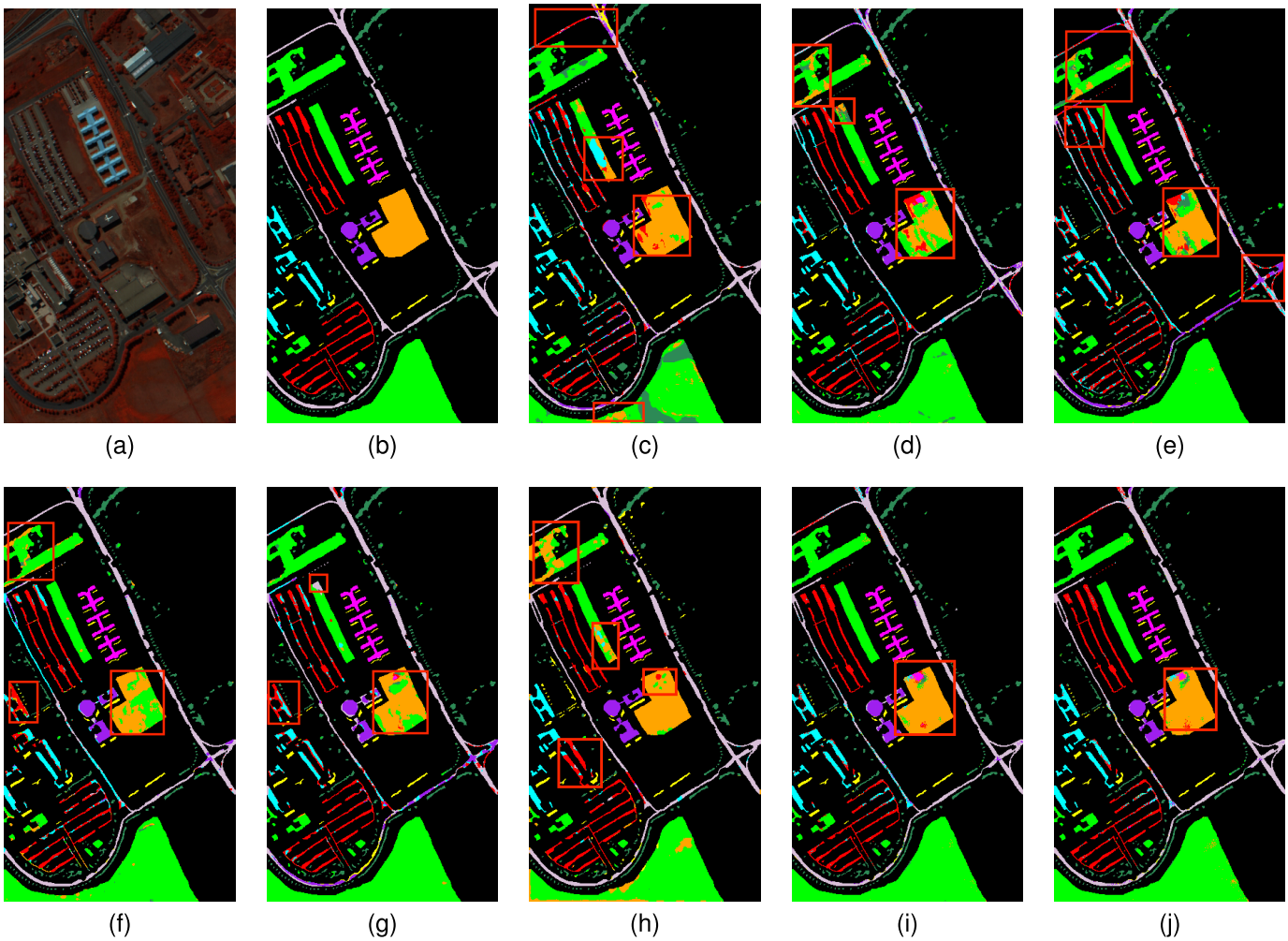


Fig. 5. Classification maps of PU dataset with various algorithms. (a) False-color image. (b) Ground truth. (c) DFSL. (d) S-DMM. (e) DCFSL. (f) CFSL. (g) CMFSL. (h) S3Net. (i) Our CGL-PNM. (j) Our CGL-PNMC. Best zoomed-in view.

to outperform CGL-PNM. Notably, CGL-PNMC shows an improvement of around 3% for IP. On the other hand, CGL-PNM has a minor edge, with an increase of 0.62% over CGL-PNMC for PU. This variance could stem from the existence of similar classes in IP (such as Corn-notill, Corn-mintill, Soybean-mintill, and Soybean-notill). These classes can be particularly tricky to distinguish due to their minimal differences between

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 23:06:40 UTC from IEEE Xplore. Restrictions apply.

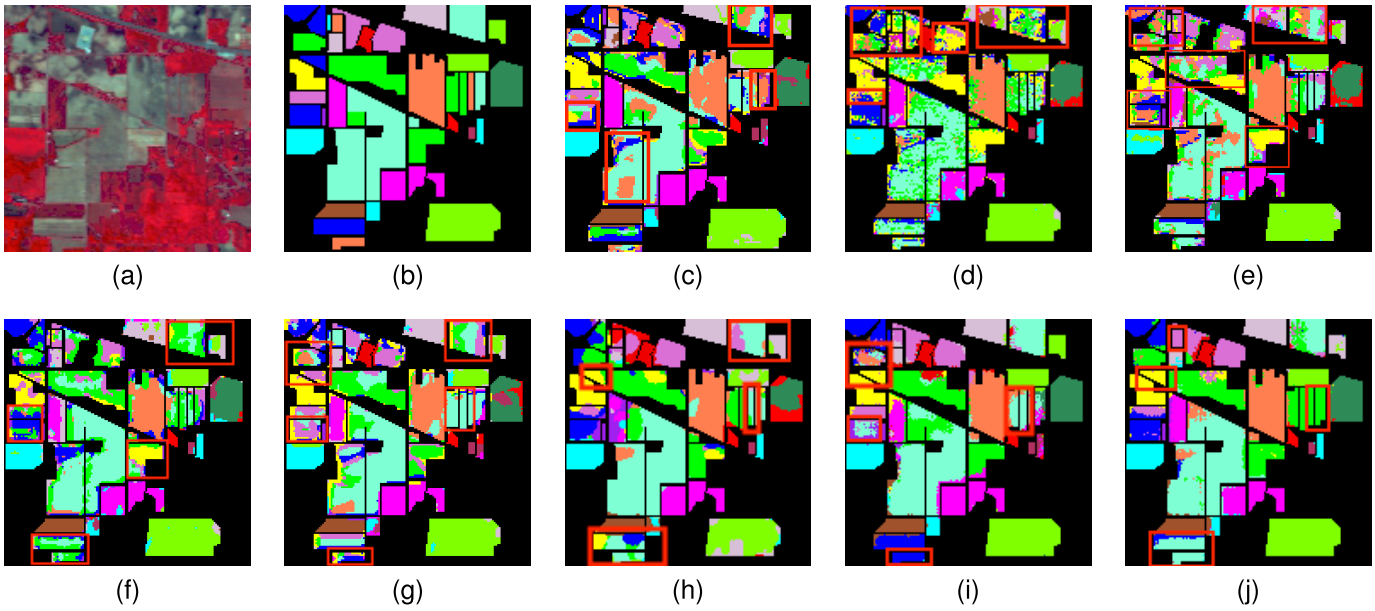


Fig. 6. Classification maps of IP dataset with various algorithms. (a) False-color image. (b) Ground truth. (c) DFSL. (d) S-DMM. (e) DCFSL. (f) CFSL. (g) CMFSL. (h) S3Net. (i) Our CGL-PNM. (j) Our CGL-PNMC. Best zoomed-in view.

them [27], [43]. The class covariance-based metric in CGL-PNMC aids in sharpening the decision boundaries of the nonlinear classifier [4].

Furthermore, we conducted a comparative experiment between our proposed fine-tuning method and an alternative fine-tuning approach. This alternative approach involves appending a fully connected layer (FCL) as a classifier to the end of the feature extractor and updating the network weight during the fine-tuning process like the approach employed in [44]. The comparison can be seen from results in R3 to R5 and R7, R10 to R12 and R14, and R17 to R19 and R21. These comparative results consistently demonstrate the superiority of our proposed method.

5) *Computational Time of Each Module:* Regarding execution time, Table VII reveals that ssMAE pretraining requires a relatively high amount of time due to the complexity of the network and the substantial number of pretraining instances. For instance, for the IP dataset, with an input patch of $9 \times 9 \times 200$, a mask size of 3×20, and an embedding dimension of 192, the number of parameters is 5.3 million.

In terms of fine-tuning, direct fine-tuning using FCL, as outlined in [44], is faster compared to alternative methods based on PNs. When comparing CGL-PNM and CGL-PNMC with PNM and PNMC, it becomes apparent that CGL-PNM and CGL-PNMC exhibit longer running times than their counterparts. This outcome was expected since both CGL-PNM and CGL-PNMC involve additional processes, such as computing the global prototype and combining global and local prototypes.

E. Comparison With Other Methods

We compared the classification performance of our proposed methods, ssMAE and CGL-PNMC, as well as ssMAE and CGL-PNM, against various specialized techniques designed to address the few-shot HSI classification challenge. The methods we benchmarked against comprise: a similarity-focused deep metric model (S-DMM) [45], a deep FSL utilizing a 3-D CNN as its central architecture (DFSL) [25], a deep cross-domain FSL anchored with a 3-D CNN (DCFSL) [31], a class-covariance metric FSL that harnesses both 3-D and 2-D convolution layers (CMFSL) [4], CFSL (an FSL method incorporating graph information for domain alignment) [11], spectral–spatial siamese network (S3Net) (a spectral–spatial twin network tailored for FSL with 1-D and 2-D convolution layers) [46], a category-specific prototype self-refinement contrastive learning (CPSRCL) [12], and a cross-domain FSL based on graph convolution contrast (GCC-FSL) [40].

To ensure a fair comparison, we reproduced the results of the benchmarked methods under the same experimental settings, adopting the hyperparameter configurations directly from each method's original research article. For each technique, we provided label information for five samples per category and tasked the model with predicting labels for the remaining samples in the test data. To maintain consistency across all methods, we recorded the indices of the labeled (training) samples and the unlabeled (test) samples, saving these indices in files that each method read from, ensuring identical data splits. We conducted five separate runs for each technique and dataset to ensure the reliability and robustness of our experimental results. The mean accuracy and covariance are detailed in Tables VIII–XI. However, due to the unavailability of the code for CPSRCL, we were unable to reproduce its results and instead directly used the results reported in its original paper. The rigorous approach allows for a more standardized comparison across different FSL methods while acknowledging limitations where they exist. To aid in direct comparison, the top three accuracy figures are highlighted as bold, underlined, and italicized, respectively. For ease of reference, ssMAE combined with CGL-PNMC is denoted as

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 23:06:40 UTC from IEEE Xplore. Restrictions apply.

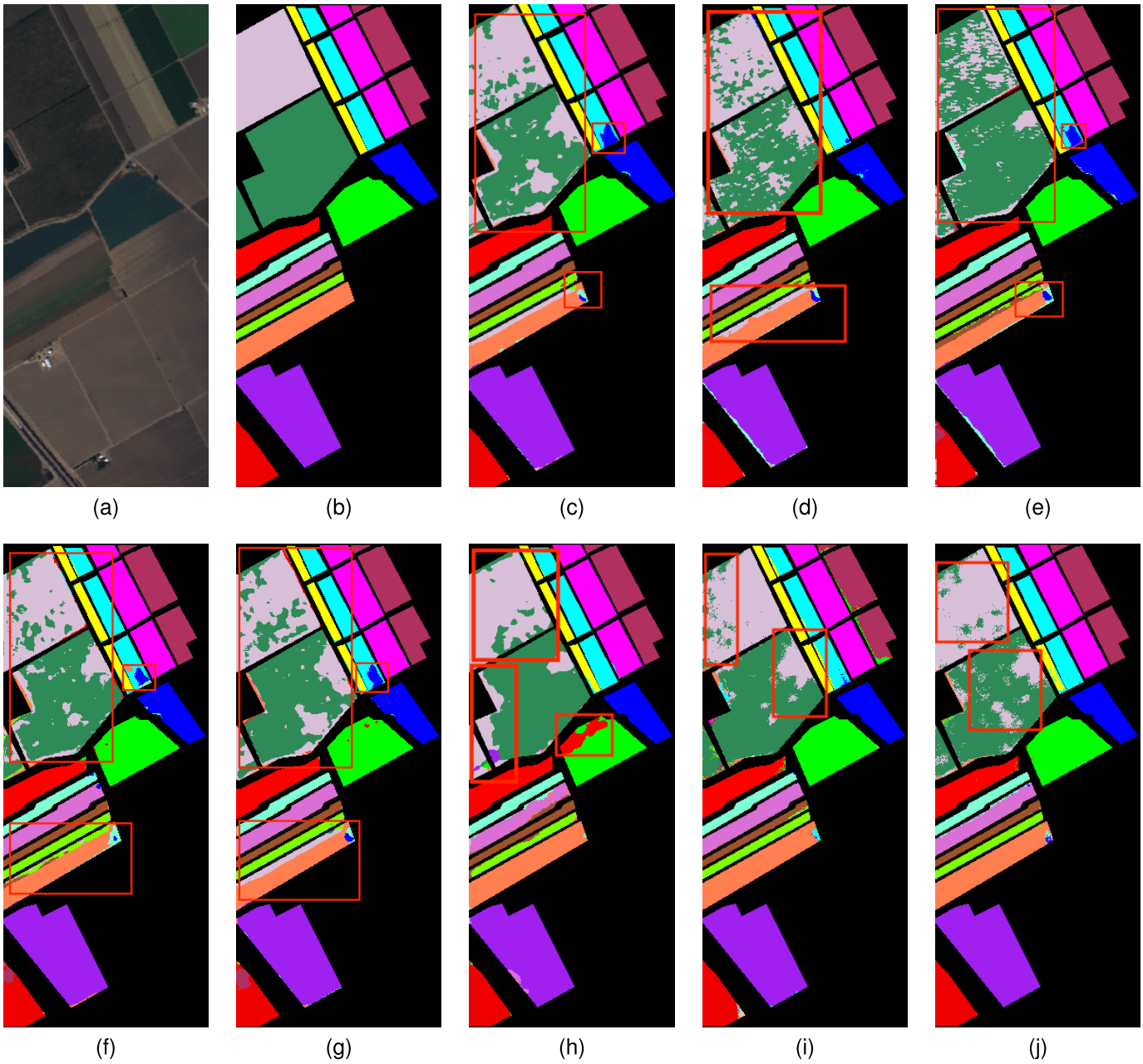


Fig. 7. Classification maps of SA dataset with various algorithms. (a) False-color image. (b) Ground truth. (c) DFSL. (d) S-DMM. (e) DCFSL. (f) CFSL. (g) CMFSL. (h) S3Net. (i) Our CGL-PNM. (j) Our CGL-PNMC. Best zoomed-in view.

CGL-PNMC, and ssMAE paired with CGL-PNM is labeled CGL-PNM.

Table VIII displays the classification results for different techniques using the PU dataset. Clearly, our proposed methods (CGL-PNMC and CGL-PNM) outperform competing approaches across all metrics. CGL-PNMC ranks as the top performer, while CGL-PNM stands as the second-best. Our proposed methods both outperform the third-best ranked method, CPSRCL, by a substantial margin: approximately 4% in OA, 3% in AA, and about 5% in Kappa metric. These findings highlight the efficacy of our proposed framework in effectively extracting discriminative features from the provided input.

Table IX presents the classification results of the discussed methods on the IP dataset. Our proposed methods exhibit superior classification performance relative to prior works. In particular, CGL-PNMC outperforms CPSRCL, which ranks third by margins of 3.24% in OA, 4.11% in AA, and 3.76% in the Kappa metric. Looking into the accuracy per class, it is noticeable that all the techniques exhibit diminished accuracy for classes 2, 3, 10, and 11. These classes, namely Corn-notil, Corn-mintill, Soybean-notil, and Soybean-mintill, bear resemblances, making them inherently difficult to distinguish, especially when confronted with notably low spatial resolution and a restricted label count of just five samples per class.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 23:06:40 UTC from IEEE Xplore. Restrictions apply.

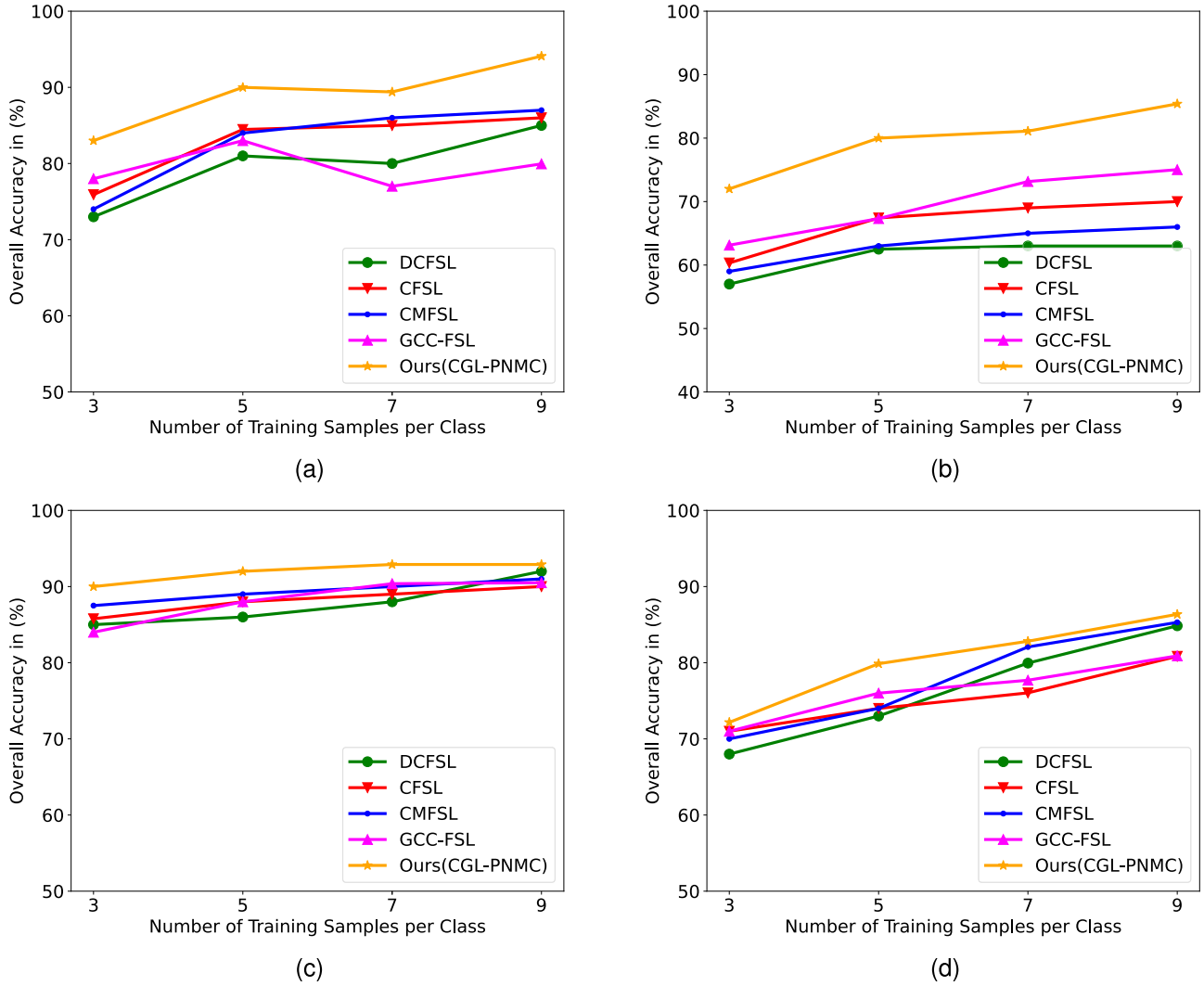


Fig. 8. OA (%) of FSL-based methods with different numbers of labeled samples on three datasets. (a) PU. (b) IP. (c) SA. (d) HO.

Table X provides an overview of the classification outcomes using the SA dataset. It is clear that our proposed methods routinely outperform the other approaches. Upon analyzing accuracy on a class-by-class basis, it becomes apparent that all methods demonstrate reduced accuracy for classes c8 and c15, labeled as "Grapes_untrained" and "Vinyard_untrained." These classes display notable similarities, posing a challenge for accurate classification. Table XI presents the results of our proposed techniques and the benchmarked methods on the HO dataset. Table XI shows that OA and AA of our method, CGL-PNM, exceed those of CMFSL by 1.19% and 1.53%.

The performance variation between CGL-PNM and CGL-PNMC can be attributed to dataset characteristics. For instance, CGL-PNMC outperforms CGL-PNM on IP and SA datasets, which contain fine-grained classes (e.g., Corn-notill versus Corn-mintill in IP, and Vineyard untrained versus Grapes untrained in SA). Fine-grained classes often exhibit subtle variations and complex feature relationships. CGL-PNMC's superior performance on these datasets can be explained by its use of Mahalanobis distance, which incorporates the data's covariance matrix. This makes it more sensitive to small, intricate differences between classes, which is crucial for fine-grained classification tasks.

For a visual comparison, we displayed the top classification maps for each of the three datasets in Figs. 5–7. In each classification map, we used bounding boxes to highlight regions where misclassifications are predominant, facilitating comparisons between classification maps. Additionally, we included composite false-color images alongside their corresponding ground truth for straightforward comparison, with reference to Table I for clarity on category-color correspondence. Fig. 5 shows that our CGL-PNM and CGL-PNMC methods generate classification maps closely aligned with the ground truth. Notably, our proposed methods achieve more accurate and compact results for the "Meadows" class in comparison to alternative approaches. In contrast, other techniques exhibit significant misclassification in the "Bare soil" class (indicated by the orange hue), whereas our methods display only minimal misclassification. These outcomes underline the reliability of our approach.

In Fig. 6, it is evident that the classification maps, derived from different methods for the IP dataset, exhibit numerous misclassifications. Clear misclassifications are predominantly in four classes: Corn-notil, Corn-mintill, Soybean-notil, and Soybean-mintill. For instance, areas marked as "Soybean-notil" are sometimes incorrectly labeled as "Soybean-mintil"

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 23:06:40 UTC from IEEE Xplore. Restrictions apply.

TABLE XII
NUMBER OF PARAMETERS AND INFERENCE TIME OF THE PROPOSED CGL-PNMC (OURS) AND OTHER METHODS

Dataset		DFSL	S-DMM	DCFSL	CFSL	CMFSL	S3Net	Ours
IP	Parms.	38,140	40,129	4,268,994	218,392	622,943	155,008	5,384,064
	Time	0.4	23.23	3.38	2.06	2.06	6.93	4.25
SA	Parms.	38,140	40,385	4,269,394	218,792	622,943	155,008	5,384,064
	Time	1.10	153.18	37.05	16.88	7.34	36.71	37.71
PU	Parms.	38,140	33,921	4,259,194	208,692	622,943	91,517	5,366,784
	Time	0.34	120.24	22.98	9.38	7.10	21.80	20.49
HO	Parms.	-	16,545	4,259,294	212,792	622,943	154,715	5,378,880
	Time		38.48	3.41	3.43	2.50	6.35	9.68

or "Corn-notil." These misclassifications can be attributed to the close similarities between these classes, complicating their differentiation. Furthermore, the IP dataset's spatial resolution is considerably lower than the PU dataset's (20 m/pixel as opposed to 1.3 m/pixel). At 20 m/pixel, each pixel corresponds to a land area of 20×20 m^2. Such a sizable region can include varied materials, leading to pronounced spectral variability within individual pixels [47].

On the other hand, the classification maps generated by all the methods for the SA dataset align closely with the ground truth, as seen in Fig. 7. The spatial resolution of the SA dataset is fivefold greater than the IP dataset, facilitating its classification. Also, from Fig. 7, it is apparent that our proposed methods, CGL-PNMC and CGL-PNM, manage to classify most classes correctly. Nevertheless, several misclassifications are still observed in two closely related classes (Vinyard_untrained versus Grapes_untrained), proving to be a challenge for both our approach and other methods.

Subsequently, we explored the impact of varying the number of labeled samples to assess the efficacy of our proposed method. Specifically, we selected three, five, seven, and nine annotated samples per class from the target domain for fine-tuning, reserving the rest for testing. Fig. 8 presents the results of our experimentation across the four datasets and various FSL methods. As expected, the OA of the different techniques improves with the increase in labeled samples. More significantly, our proposed CGL-PNMC consistently outperforms other approaches. These findings validate the robustness of CGL-PNMC in adapting to changes in the number of labeled samples.

Additionally, like [4], we also compare the number of parameters (Parms.) and inference time in seconds of the proposed CGL-PNMC (ours) with other methods. The results are shown in Table XII. We left the result for the DFSL method on HO dataset blank because this method did not share its code for pretraining, only providing its pretraining model for IP, SA, and PU datasets. From Table XII, it can be observed that DFSL and CMFSL have the same number of parameters across different datasets due to their use of band selection preprocessing techniques; resulting in the same number of input bands. In contrast, our method and the rest of the compared methods have varying numbers of parameters for different datasets owing to variations in the number of input bands.

Despite having the most parameters among the compared methods, our method achieves faster inference times than S-DMM and S3Net. This efficiency can be attributed to the efficient inference process of our network that directly compares the extracted features with fixed prototypes. In S-DMM, after feature extraction, the test samples are processed using a relational network, which involves concatenating features, multiple convolutional layers, and a function to compute the relationship between samples. This additional process increases computational time. In S3Net, which is based on the Siamese network strategy, the network needs to compare the test samples with each training sample. This pairwise comparison increases computational time significantly.

V. CONCLUSION

In this article, we propose novel pretraining and fine-tuning strategies to address few-shot classification problems. First, ssMAE, an SSL strategy, exploits information from unlabeled samples to learn their general features in the pretraining stage. The ssMAE employs spatiospectral mask sampling that not only structurally fits the HSI input structure but is also more effective than spatial-only and spectral-only mask samplings. Additionally, unlike general MAE with a transformer backbone that employs a linear layer as initial feature embedding, we replaced the linear layer with a 3-D convolutional layer as it is more effective in extracting local features from visible 3-D sub-patches. Second, we fine-tuned the feature extractor, ssMAE encoder, by exploiting the few-shot annotated labels. To minimize the impact of subpar episodes, we propose a hybrid episode learning strategy that combines global and local prototypes (CGL prototype) and exploits the labeled training samples while still paying more attention to specific samples (support and query sets). Third, the hybrid episode learning strategy is implemented in two types of PN: PNM and PNMC. Our ablation studies show that this strategy improves the performance of both PNM and PNMC. The experiments demonstrate that our proposed framework, pretraining using ssMAE and fine-tuning using hybrid episode strategy (CGL-PNM and CGL-PNMC), achieves superior results to other alternative few-shot methods.

REFERENCES

[1] W. N. Khotimah, M. Bennamoun, F. Boussaid, F. Sohel, and D. Edwards, "A high-performance spectral–spatial residual network for hyperspectral image classification with small training data," *Remote Sens.*, vol. 12, no. 19, p. 3137, Sep. 2020.

[2] S. K. Roy, G. Krishna, S. R. Dubey, and B. B. Chaudhuri, "HybridSN: Exploring 3-D-2-D CNN feature hierarchy for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 2, pp. 277–281, Feb. 2019.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 23:06:40 UTC from IEEE Xplore. Restrictions apply.

[3] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[4] B. Xi, J. Li, Y. Li, R. Song, D. Hong, and J. Chanussot, "Few-shot learning with class-covariance metric for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 31, pp. 5079–5092, 2022.

[5] B. Fang, Y. Li, H. Zhang, and J. C.-W. Chan, "Collaborative learning of lightweight convolutional neural network and deep clustering for hyperspectral image semi-supervised classification with limited training samples," *ISPRS J. Photogramm. Remote Sens.*, vol. 161, pp. 164–178, Mar. 2020.

[6] M. E. Paoletti, J. M. Haut, J. Plaza, and A. Plaza, "Deep learning classifiers for hyperspectral imaging: A review," *ISPRS J. Photogramm. Remote Sens.*, vol. 158, pp. 279–317, Dec. 2019.

[7] A. Mishra, U. P. Singh, and K. P. Singh, "A lightweight relation network for few-shots classification of hyperspectral images," *Neural Comput. Appl.*, vol. 35, no. 15, pp. 11417–11430, May 2023.

[8] J. Xia, Y. Cui, W. Li, L. Wang, and C. Wang, "Lightweight self-attention residual network for hyperspectral classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[9] C. Cao, H. Xiang, W. Song, H. Yi, F. Xiao, and X. Gao, "Lightweight multiscale neural architecture search with spectral–spatial attention for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5505315.

[10] X. Li, M. Ding, and A. Pižurica, "Deep feature fusion via two-stream convolutional neural network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 4, pp. 2615–2629, Apr. 2020.

[11] Y. Zhang, W. Li, M. Zhang, S. Wang, R. Tao, and Q. Du, "Graph information aggregation cross-domain few-shot learning for hyperspectral image classification," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 2, pp. 1912–1925, Feb. 2024, doi: 10.1109/TNNLS.2022.3185795.

[12] Q. Liu, J. Peng, N. Chen, W. Sun, Y. Ning, and Q. Du, "Category-specific prototype self-refinement contrastive learning for few-shot hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5524416.

[13] L. Jing and Y. Tian, "Self-supervised visual feature learning with deep neural networks: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 11, pp. 4037–4058, Nov. 2021.

[14] D. Ibañez, R. Fernandez-Beltran, F. Pla, and N. Yokoya, "Masked auto-encoding spectral–spatial transformer for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5542614.

[15] L. Huang, Y. Chen, and X. He, "Spectral–Spatial masked transformer with supervised and contrastive learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023, Art. no. 5508718.

[16] S. Jia, S. Jiang, Z. Lin, N. Li, M. Xu, and S. Yu, "A survey: Deep learning for hyperspectral image classification with few labeled samples," *Neurocomputing*, vol. 448, pp. 179–204, Aug. 2021.

[17] N. Wambugu et al., "Hyperspectral image classification on insufficient-sample and feature learning using deep neural networks: A review," *Int. J. Appl. Earth Observ. Geoinf.*, vol. 105, Dec. 2021, Art. no. 102603.

[18] J. Peng, W. Sun, L. Ma, and Q. Du, "Discriminative transfer joint matching for domain adaptation in hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 6, pp. 972–976, Jun. 2019.

[19] J. T. Zhou, I. W. Tsang, S. J. Pan, and M. Tan, "Multi-class heterogeneous domain adaptation," *J. Mach. Learn. Res.*, vol. 20, no. 57, pp. 1–31, Apr. 2019.

[20] N. A. A. Braham, L. Mou, J. Chanussot, J. Mairal, and X. X. Zhu, "Self supervised learning for few shot hyperspectral image classification," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2022, pp. 267–270.

[21] J. Yue, L. Fang, H. Rahmani, and P. Ghamisi, "Self-supervised learning with adaptive distillation for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5501813.

[22] J. Yang, L. Xiao, Y.-Q. Zhao, and J. C.-W. Chan, "Unsupervised deep tensor network for hyperspectral-multispectral image fusion," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 9, pp. 13017–13031, Sep. 2024, doi: 10.1109/TNNLS.2023.3266038.

[23] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 16000–16009.

[24] W. N. Khotimah, M. Bennamoun, F. Boussaid, L. Xu, D. Edwards, and F. Sohel, "MCE-ST: Classifying crop stress using hyperspectral data with a multiscale conformer encoder and spectral-based tokens," *Int. J. Appl. Earth Observ. Geoinf.*, vol. 118, Apr. 2023, Art. no. 103286.

[25] B. Liu, X. Yu, A. Yu, P. Zhang, G. Wan, and R. Wang, "Deep few-shot learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 4, pp. 2290–2304, Apr. 2019.

[26] C. Zhang, J. Yue, and Q. Qin, "Global prototypical network for few-shot hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 4748–4759, 2020.

[27] Z. Ji, X. Chai, Y. Yu, Y. Pang, and Z. Zhang, "Improved prototypical networks for few-shot learning," *Pattern Recognit. Lett.*, vol. 140, pp. 81–87, Dec. 2020.

[28] B. N. Oreshkin, P. Rodríguez, and A. Lacoste, "TADAM: Task dependent adaptive metric for improved few-shot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., Jan. 2018, pp. 1–18.

[29] P. Bateni, R. Goyal, V. Masrani, F. Wood, and L. Sigal, "Improved few-shot visual classification," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 14493–14502.

[30] J. Snell, K. Swersky, and R. Zemel, "Prototypical networks for few-shot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, I. Guyon et al., Ed., 2017, pp. 1–11.

[31] Z. Li, M. Liu, Y. Chen, Y. Xu, W. Li, and Q. Du, "Deep cross-domain few-shot learning for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2019.

[32] S. Laenen and L. Bertinetto, "On episodes, prototypical networks, and few-shot learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. S. Liang, and J. W. Vaughan, Eds., Jan. 2020, pp. 24581–24592.

[33] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.

[34] A. Vaswani, "Attention is all you need," in *Proc. 31st Conf. Neural Inf. Process. Syst.*, vol. 30, Dec. 2017, pp. 5999–6009.

[35] S. C. Wong, A. Gatt, V. Stamatescu, and M. D. McDonnell, "Understanding data augmentation for classification: When to warp?" in *Proc. Int. Conf. Digit. Image Computing: Techn. Appl. (DICTA)*, Nov. 2016, pp. 1–6.

[36] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "DeepLab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected CRFs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 4, pp. 834–848, Apr. 2017.

[37] H.-X. Dou, X.-S. Lu, C. Wang, H.-Z. Shen, Y.-W. Zhuo, and L.-J. Deng, "PatchMask: A data augmentation strategy with Gaussian noise in hyperspectral images," *Remote Sens.*, vol. 14, no. 24, p. 6308, Dec. 2022.

[38] J. Li, X. Li, and Y. Yan, "Unlocking the potential of data augmentation in contrastive learning for hyperspectral image classification," *Remote Sens.*, vol. 15, no. 12, p. 3123, Jun. 2023.

[39] C. Debes et al., "Hyperspectral and LiDAR data fusion: Outcome of the 2013 GRSS data fusion contest," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2405–2418, Jun. 2014.

[40] Z. Ye, J. Wang, T. Sun, J. Zhang, and W. Li, "Cross-domain few-shot learning based on graph convolution contrast for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024, Art. no. 5504614.

[41] C. Feichtenhofer, H. Fan, Y. Li, and K. He, "Masked autoencoders as spatiotemporal learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, Jan. 2022, pp. 35946–35958.

[42] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Hum. Lang. Technol.*, Jun. 2019, pp. 4171–4186.

[43] M. Nauta, R. van Bree, and C. Seifert, "Neural prototype trees for interpretable fine-grained image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 14933–14943.

[44] W. Li, Q. Liu, Y. Wang, and H. Li, "Transfer learning with limited samples for the same source hyperspectral remote sensing images classification," *Int. Arch. Photogramm., Remote Sens. Spatial Inf. Sci.*, vol. 43, pp. 405–410, May 2022.

[45] B. Deng, S. Jia, and D. Shi, "Deep metric learning-based feature embedding for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 2, pp. 1422–1435, Feb. 2020.

[46] Z. Xue, Y. Zhou, and P. Du, "S3Net: Spectral–Spatial Siamese network for few-shot hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5531219.

[47] R. A. Borsoi et al., "Spectral variability in hyperspectral data unmixing: A comprehensive review," *IEEE Geosci. Remote Sens. Mag.*, vol. 9, no. 4, pp. 223–270, Dec. 2021.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 23:06:40 UTC from IEEE Xplore. Restrictions apply.